Grading Criteria:

Part 1: Working Program (50% of Assignment)

This portion accounts for 50% of the assignment mark (= 7.5% of overall course grade).

Criteria	Weight	Assessment Criteria
Correctness & Functionality	20%	- Implements Othello rules correctly (initial board, flipping logic, valid move checks, passing, game end).
		- Accurately detects a winner or tie.
		- Handles edge cases (board boundaries, no available moves).
Minimax (or AI) Implementation	10%	- Demonstrates a working AI using minimax or alpha-beta pruning.
		- Includes a reasonable evaluation function (piece count, mobility, etc.).
		- Chooses moves logically and integrates seamlessly with the game loop.
Design & Use of Arrays	10%	- Uses basic arrays (char[][], etc.) to represent the board.
		- Code organization with clear, modular methods.
		- Properly separates concerns (e.g., one class or set of methods for board, another for AI logic).
User Interface & Polish	10%	- Command-line prompts are clear and user-friendly.
		- Error handling for invalid inputs (out-of-range row/column).
		- Board display is neat, helps user see updates after each move.

Part 2: Individual Interview (50% of Assignment)

This portion accounts for the remaining 50% of the assignment mark (= 7.5% of overall course grade).

Each student will have a short **oral defense** of the code. You will be asked about your specific responsibilities and must demonstrate knowledge in **all** relevant areas. The table below outlines areas that will be evaluated:

Topic (Weight)	Responsibilities	Full Mastery (9–10 points)	Partial Understanding (5–8 points)	Minimal Understanding (1–4 points)	No Understanding (0 points)
1. Board Representation & Initialization (10%)	- Use a suitable data structure (e.g., 2D array); Set correct starting positions (two black and two white in the center); Provide helper methods (display board, reset, piece counts)	- Clearly explains array choice/size; Demonstrates standard Othello setup; Justifies helper methods; Answers follow-up questions confidently	- Basic explanation of the array and initial setup; Struggles with edge cases (e.g., resizing the board)	- Very limited clarity on board structure; Omits key details like how initial pieces are placed	- Cannot describe data structure or how the board is initialized
2. Move Validation & Stone Flipping Logic (10%)	- Check legality of moves (must flip at least one opponent piece); Flip pieces in valid directions (horizontal, vertical, diagonal); Optionally track valid moves or flip counts	- Clearly describes multi-directional flip-check logic; Shows how valid moves are verified and pieces flipped; Handles boundaries; Answers questions well	- Understands flipping basics but is fuzzy on certain directions or boundary conditions	- Knows flipping is needed but cannot detail the logic; Cannot confirm correctness	- No explanation of how moves are checked or flipped
3. Game Flow & State Management (10%)	- Manage turn order (which player is next); Handle “pass” when no valid moves exist; Detect end-of-game (no moves for both players); Determine winner by piece counts	- Clearly explains turn order; Manages passes properly; Knows when the game ends; Understands final scoring; Covers corner cases (e.g., consecutive passes)	- Basic turn logic but uncertain about repeated passes or other edge cases	- Vague explanation of turn order or passing; Does not describe end-of-game conditions	- No concept of how or when the game finishes; Cannot explain turn order or passing

4. AI (Minimax or Alpha-Beta) Implementation (10%)	- Implement minimax or alpha-beta search; Define a board-evaluation function or heuristic; Handle search depth or pruning (if applicable)	- Correctly describes minimax/alpha-beta recursion; Demonstrates evaluation function; Can walk through an AI decision; Explains performance considerations; Answers follow-ups	- Understands the idea of exploring future moves but struggles with alpha-beta details or heuristic design	- Only vague references to AI logic; Relies on guesswork over a structured algorithm	- Cannot describe minimax, alpha-beta, or any AI approach
5. User Interface & Integration / Testing (10%)	- Provide a command-line or similar UI; Integrate board, flipping logic, and AI; Develop tests/scenarios for correct moves, AI decisions, and end-of-game states	- Presents clear, user-friendly prompts; Explains how modules interact; Shows thorough testing with multiple scenarios; Confidently answers integration questions	- Functional UI but minimal testing strategy or weak integration details	- UI is present but not well-designed or tested; Struggles to explain how different parts fit together	- No explanation of user interaction or testing; Cannot show how features are integrated